

Inverting the Formalization Workflow: Prototyping an MPC Protocol in Rocq with an LLM Agent

Cheng-Hui Weng
Nagoya University

Abstract

Formalization traditionally comes last, and this creates a high barrier when a construction depends on unfamiliar mathematics. We instead use formalization as the first step and treat it as a prototyping medium. In this workflow, an LLM agent drafts formal models, a theorem prover checks whether they stand on solid logical ground, and the human steers the process while learning from the output. We report on an ongoing 42-file Rocq formalization of SMC-PGG, a covering-space-based MPC protocol that spans five mathematical domains learned through formalization itself. The paper closes with observations about human judgment, LLM breadth, and prover-backed certainty.

Keywords. LLM-assisted formalization, Rocq, interactive theorem proving, secure multi-party computation, covering spaces.

Introduction. Formalization traditionally comes last. A researcher studies the relevant mathematics, designs a construction informally, and only then translates the result into a proof assistant as a final verification step. This ordering works well when the mathematical landscape is already familiar, but it becomes a serious obstacle when the construction draws on several domains that the researcher has not yet studied. In this work, we reverse the order and use formalization as a prototyping medium. An LLM agent¹ drafts formal models from natural-language direction, and a theorem prover checks whether each draft stands on solid logical ground. In this setting, the human learns from the structured output of each iteration. Definitions that type-check reveal useful abstractions, and failed proof obligations reveal missing assumptions.

This extended abstract makes three case-study contributions. First, it presents an inverted workflow in which formalization precedes domain mastery, following the pattern $\text{idea} \rightarrow \text{formalize} \rightarrow \text{learn}$, so that unfamiliar mathematical ideas can be tested early on solid logical ground. Second, it reports on an ongoing 42-file Rocq formalization of SMC-PGG, a covering-space-based MPC protocol. Third, it draws methodological observations about the distinct roles of human judgment, LLM breadth, and prover-backed certainty.

Inverted Workflow. The starting point was a hypothesis from separate ongoing work on generalizing secure multi-party computation (MPC). The hypothesis was that any surjective relation can serve as a secret-sharing mechanism if legitimate parties possess a trapdoor for reconstruction. Covering spaces in algebraic topology fit this pattern. The fiber over a basepoint can represent the secret, the monodromy action can represent the computation, and the covering map can represent reconstruction. However, even formulating such connections informally already requires time to understand each contributing domain, and developing them to the level of a publishable pen-and-paper argument takes much longer. Rather than waiting to master group theory, algebraic geometry, and coding theory in advance, we used formalization in Rocq/MathComp to test whether this connection makes sense on solid logical ground.

From that point on, formalization became a form of exploration. The human did not need to master every required domain before starting. Instead, the LLM contributed definitions, lemma statements, and proof strategies, while the theorem prover determined which attempts could stand as part of a coherent development. Three roles then became clear. The human provided judgment about direction, abstraction, and axiomatization boundaries. The LLM provided breadth across domains. The theorem prover provided certainty. When the development compiled without **Admitted**, its logical structure held relative to the stated axioms.

¹All LLM-assisted work reported here used Claude (Anthropic), specifically Claude Opus 4.6.

Case Study. To test this workflow, we applied it to SMC-PGG (Secure Multi-party Computation via Parametric Geometry Group). At the protocol level, a monodromy representation $\rho : G \rightarrow S_N$ maps group elements to permutations on N sheets and hides a secret in the fiber structure of a covering space. This layer is packaged as an HB mixin (`MonodromyReprType`) together with session-typed programs for dealing, computing, and reconstructing. The next question concerned the adversary’s search space. Here the key point is that computation histories that differ only by reordering commuting actions should be indistinguishable to the adversary. Right-Angled Artin Group (RAAG) trace equivalence provides the natural formal language for identifying such histories. The Cartier-Foata theorem, proved from MathComp primitives alone, then supplied exact trace counts through a clique polynomial recurrence. This illustrates learning through formalization, since the LLM introduced Cartier-Foata when the proof required counting distinct computation histories.

The reconstruction layer instantiated a threshold-scheme interface using Massey’s secret sharing from linear codes. Hyperelliptic algebraic-geometry codes provided parametric (k, T) -thresholds through resultant computations. An `AlgebraicRigidity` record then related one algebraic choice to four quantities: search-space complexity, the number of admissible traces, the security bound, and the threshold gap. Finally, the security layer established a collusion bound, namely total variation distance $\leq \varepsilon + 2(T-1)/N$, where ε is a privacy parameter, T is the reconstruction threshold, and N is the number of sheets. It also established Grover mitigation and an abelian collapse theorem, showing that in the regular abelian setting, non-abelian groups are necessary for non-trivial security.

The formalization spans 42 Rocq files and touches five domains: group theory, combinatorics, algebraic geometry, coding theory, and information theory. We entered the project with experience in protocol design, but the other four domains were learned during the formalization process. The remaining axioms are concentrated at the algebraic-geometry interface. They concern evaluation-map rank and encoding assumptions, monodromy/code compatibility, multiplicative structure, and the genus-0 PGL automorphism bound.

Observations. This case study yields three methodological lessons. The workflow succeeded because the feedback loop was tight. The LLM produced drafts, the theorem prover accepted or rejected them, and the human used those results both to steer the next step and to learn domain knowledge on demand. The workflow also had clear limits. Architectural decisions and axiomatization boundaries required human judgment, and dependent-type unification failures required human-level diagnosis. Even so, the balance was productive because the human could focus on judgment rather than tactic syntax.

Related Work. Related work on LLM-assisted proving includes whole-proof generation [1], informal-to-formal translation [2], and retrieval-augmented search [3]. It also includes IDE integration [4] and reinforcement-learning-based training [5]. Here, by contrast, the LLM primarily serves as a mechanism for domain-knowledge transfer in a human-directed exploratory workflow.

Conclusion. This case study suggests a methodological claim about AI-assisted formalization. An LLM agent can be useful not only for proof search, but also for cross-domain exploration in an ongoing research project. In that setting, formalization need not be the final verification step. It can instead serve as a prototyping medium that tests whether a mathematical idea survives contact with a theorem prover while simultaneously teaching the surrounding domain. The traditional order of learn $\rightarrow$ design $\rightarrow$ formalize can therefore be inverted into idea $\rightarrow$ formalize $\rightarrow$ learn.

References

- [1] E. First, M. N. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *ESEC/FSE 2023*, 2023.
- [2] A. Jiang et al. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *ICLR 2023*, 2023.
- [3] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS 2023 (Datasets and Benchmarks)*, 2023.
- [4] P. Song, K. Yang, and A. Anandkumar. Lean Copilot: LLMs as copilots for theorem proving in Lean. *arXiv:2404.12534*, 2024.
- [5] Google DeepMind. AlphaProof and AlphaGeometry 2. Technical report, 2024.
- [6] G. Barthe et al. Computer-aided security proofs for the working cryptographer. In *CRYPTO 2011*, LNCS 6841, Springer, 2011.